Práctica demostrativa 2 — Estilos con CSS

PASO 0 · Antes de escribir una sola regla (5 min)

PASO 1 · Vincular la hoja externa (5 min)

PASO 2 · El reinicio y el modelo de caja (15 min)

PASO 3 · Selectores y especificidad (15 min)

PASO 4 · Variables y tipografía (10 min)

PASO 5 · Estructura y espaciado de la página (10 min)

PASO 6 · Flexbox en la navegación (15 min)

PASO 7 · Tarjetas con Flexbox (10 min)

PASO 8 · Tabla, formulario y estados (10 min)

CIERRE · Lista de verificación (5 min)

Errores más comunes y cómo se detectan

Ejercicio inmediato (trabajo del estudiante)

Lo que sigue

Anexo · Hoja de estilos completa

Aplicaciones Web · E-APW-2 · Unidad II. Desarrollo de Front End Modalidad: demostración guiada en laboratorio · **Duración:** 90 minutos **Punto de partida:** la página `index.html` construida en la práctica demostrativa 1

Propósito

Dar estilo a la página de la Semana de la Ingeniería sin modificar su contenido: vincular una hoja externa, controlar selectores y especificidad, dominar el modelo de caja y acomodar elementos con Flexbox.

Igual que en la práctica anterior, el docente escribe y el grupo replica. Cada bloque de reglas se guarda y se recarga antes de continuar.

Al terminar, el estudiante será capaz de

- Vincular una hoja de estilos externa y explicar por qué se prefiere a las otras dos formas.
- Predecir qué regla gana cuando dos reglas afectan al mismo elemento.
- Explicar qué hace `box-sizing: border-box` y por qué se aplica desde el inicio.
- Acomodar elementos en una fila con Flexbox y distribuir el espacio disponible.

Preparación previa

Requisito	Verificación
La página de la práctica 1 terminada	`index.html` abre y valida sin errores
Visual Studio Code con Live Server	Aparece *Go Live*
Navegador con herramientas de desarrollador	F12 abre el panel

Nota para el docente. La pestaña *Elements* de las herramientas de desarrollador debe estar visible durante toda la sesión. En CSS, buena parte del aprendizaje consiste en leer qué regla se aplicó, cuál quedó tachada y por qué.

PASO 0 · Antes de escribir una sola regla (5 min)

Abrir la página tal como quedó en la práctica anterior. **Ya tiene estilos**, aunque nadie los haya escrito.

Abrir las herramientas de desarrollador, seleccionar el `<h1>` y mostrar el panel de estilos: aparece `user agent stylesheet` con un tamaño de fuente y un margen.

> **Idea que hay que dejar clara desde el principio.** El navegador trae su propia hoja de estilos. Escribir CSS no es “agregar diseño a algo que no tenía”: es **sustituir** las decisiones que el navegador ya tomó por las nuestras.

Las tres formas de aplicar CSS:

Forma	Sintaxis	Cuándo se usa
En línea	`<p style="color: red">`	Prácticamente nunca. Mezcla contenido y presentación
Interna	`<style>` dentro del `<head>`	Pruebas rápidas y archivos de un solo uso
Externa	Archivo `.css` vinculado	**Siempre.** Una sola hoja da estilo a todo el sitio

---

PASO 1 · Vincular la hoja externa (5 min)

1. Crear la carpeta `css` y dentro el archivo `estilos.css`.
2. En el `<head>` del HTML, **después** del `<title>`:

```
        <link rel="stylesheet" href="css/estilos.css">
```

3. Escribir la primera regla, solo para comprobar la conexión:

```
        body {
            background-color: #f4f6f7;
        }
```

Guardar y recargar: el fondo cambia. La hoja está conectada.

> **Si no cambia nada**, el problema casi siempre es la ruta. Revisar que la carpeta se llame `css`, que el archivo se llame `estilos.css` y que todo vaya en minúsculas. Es el mismo error de rutas que apareció con las imágenes en la práctica anterior.

PASO 2 · El reinicio y el modelo de caja (15 min)

Este es el paso conceptual más importante de la sesión.

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
```

Qué es el modelo de caja. Todo elemento HTML es una caja con cuatro capas, de adentro hacia afuera: **contenido**, **padding** —relleno interior—, **border** —borde— y **margin** —separación exterior—.

Por qué `border-box`. Antes de explicarlo, hacer la demostración:

```
.demo {
    width: 300px;
    padding: 20px;
    border: 5px solid #333;
    background: #cde;
}
```

Aplicar temporalmente esa clase a un párrafo, seleccionarlo en las herramientas de desarrollador y leer el ancho real que reporta el navegador: **350 píxeles**, no 300. El padding y el borde se suman por fuera.

Ahora activar `box-sizing: border-box` y volver a medir: **300 píxeles exactos**. El padding y el borde quedaron incluidos dentro del ancho declarado.

> **La consecuencia práctica.** Sin esa línea, construir una rejilla de tres columnas de 33% cada una se rompe en cuanto se agrega padding. Con ella, 300 píxeles son 300 píxeles. Por eso el reinicio va al principio de toda hoja de estilos y no se discute.

Eliminar la clase `.demo` antes de continuar.

PASO 3 · Selectores y especificidad (15 min)

Escribir estas cuatro reglas **en este orden**, y antes de recargar preguntar al grupo de qué color quedará el párrafo:

```css
p {
    color: green;
}

.destacado {
    color: blue;
}

#principal {
    color: red;
}
```

Con un párrafo así en el HTML:

```html
<p class="destacado" id="principal">¿De qué color soy?</p>
```

El resultado es **rojo**. Después mostrar en las herramientas de desarrollador cómo las otras dos reglas aparecen **tachadas**: el navegador las evaluó y las descartó.

La jerarquía de especificidad, de menor a mayor:

Selector	Ejemplo	Peso
Elemento	`p`	Bajo
Clase, atributo, pseudoclase	`.destacado`, `a:hover`	Medio
Identificador	`#principal`	Alto
Estilo en línea	`style="..."`	Muy alto
`!important`	`color: red !important`	Gana sobre todo

Cuando dos reglas tienen la misma especificidad, gana la última escrita. Eso es la cascada.

Otros selectores que se usarán hoy:

```css
nav a { }              /* enlaces dentro de la navegación (descendiente) */
.tarjetas > article { } /* articles hijos directos del contenedor */
input[type="text"] { } /* por atributo */
a:hover { }            /* pseudoclase: estado del elemento */
```

> **Regla del curso.** Trabajar con clases. Los identificadores se reservan para anclas y para JavaScript, y **`!important` no se usa**. Quien necesita `!important` es porque perdió el control de su hoja de estilos, y en un proyecto de equipo eso se vuelve inmanejable en semanas.

Borrar las tres reglas de la demostración antes de seguir.

PASO 4 · Variables y tipografía (10 min)

```css
:root {
    --color-principal: #0d5c63;
    --color-oscuro: #0a3a3f;
    --color-texto: #24333a;
    --color-claro: #f2f8f8;
    --color-borde: #c9dcde;
    --espacio: 1rem;
    --fuente: Arial, Helvetica, sans-serif;
}

body {
    font-family: var(--fuente);
    font-size: 1rem;
    line-height: 1.6;
    color: var(--color-texto);
    background-color: #ffffff;
}

h1, h2, h3 {
    line-height: 1.25;
    margin-bottom: 0.5rem;
}

h1 { font-size: 2rem; }
h2 { font-size: 1.5rem; color: var(--color-principal); }
h3 { font-size: 1.15rem; }

p, ul, ol {
    margin-bottom: var(--espacio);
}
```

Qué es una variable CSS. Un valor con nombre, declarado una vez en `:root` y reutilizado con `var()`. Cambiar la identidad visual completa del sitio se vuelve cuestión de editar cinco líneas.

> **Demostración.** Cambiar `--color-principal` a naranja y recargar: todos los títulos cambian de golpe. Volver al valor original. En el proyecto integrador, esto es exactamente lo que permite que diez módulos compartan la misma identidad.

Sobre `rem` y no `px`. `1rem` equivale al tamaño de fuente base del navegador. Si un usuario aumenta ese tamaño desde la configuración —algo que mucha gente hace—, el texto en `rem` responde y el texto en `px` no. Es accesibilidad, no preferencia.

PASO 5 · Estructura y espaciado de la página (10 min)

```css
header {
    background-color: var(--color-principal);
    color: #ffffff;
    padding: 2rem 1.5rem;
}

header p {
    margin-bottom: 0;
    color: var(--color-claro);
}

main {
    max-width: 1100px;
    margin: 0 auto;
    padding: 2rem 1.5rem;
}

section {
    margin-bottom: 3rem;
}

aside {
    background-color: var(--color-claro);
    border-left: 4px solid var(--color-principal);
    padding: 1.5rem;
    max-width: 1100px;
    margin: 0 auto 2rem auto;
}

footer {
    background-color: var(--color-oscuro);
    color: #ffffff;
    padding: 1.5rem;
    text-align: center;
}
```

La línea que hay que explicar con calma: `margin: 0 auto`. El valor `auto` en los márgenes laterales reparte el espacio sobrante en partes iguales, lo que **centra** un elemento que tiene ancho máximo. Es la forma estándar de centrar un contenedor.

> **Demostración.** Quitar `max-width` del `<main>` y recargar con la ventana maximizada: las líneas de texto se vuelven larguísimas y cuesta leerlas. Restaurarlo. Limitar el ancho de lectura es una decisión de legibilidad, no de estética.

PASO 6 · Flexbox en la navegación (15 min)

```
        nav {
            background-color: var(--color-oscuro);
            padding: 0 1.5rem;
        }

        nav ul {
            list-style: none;
            margin: 0;
            padding: 0;
            display: flex;
            gap: 1.5rem;
        }

        nav a {
            display: block;
            padding: 1rem 0;
            color: #ffffff;
            text-decoration: none;
        }

        nav a:hover {
            color: #9fd3d8;
        }
```

Antes de escribir `display: flex`, recargar con las reglas anteriores: los elementos de la lista siguen uno debajo de otro. Agregar la línea y recargar: quedan en fila. Ese contraste es la demostración.

Las propiedades del contenedor flexible:

Propiedad	Qué controla	Valores frecuentes
`display: flex`	Activa el modo flexible	—
`flex-direction`	Dirección del eje principal	`row`, `column`
`justify-content`	Distribución en el eje principal	`flex-start`, `center`, `space-between`, `space-around`
`align-items`	Alineación en el eje transversal	`flex-start`, `center`, `stretch`
`gap`	Separación entre elementos	Cualquier medida
`flex-wrap`	Permite el salto de línea	`nowrap`, `wrap`

> **Demostración en vivo, sin tocar el editor.** En las herramientas de desarrollador, seleccionar el `<ul>` y cambiar el valor de `justify-content` entre `flex-start`, `center`, `space-between` y `space-around`, recargando nada. El grupo ve el efecto de cada valor en el momento.
>
> Después **recargar la página**: todos los cambios desaparecen. Lo que se edita en el navegador es temporal; lo que se guarda en el archivo es permanente. Conviene que esto quede claro hoy, porque es una confusión que cuesta horas más adelante.

Sobre `text-decoration: none` y el `display: block` en los enlaces. El primero quita el subrayado; el segundo hace que el área sensible al clic ocupe todo el alto del padding, en lugar de solo el texto. En un teléfono, esa diferencia decide si el dedo acierta o no.

PASO 7 · Tarjetas con Flexbox (10 min)

Este paso requiere **un cambio en el HTML**: envolver los dos `<article>` de la sección de talleres.

```
        <div class="tarjetas">
            <article class="tarjeta"> ... </article>
            <article class="tarjeta"> ... </article>
        </div>
```

> **Momento didáctico.** Aquí sí corresponde un `<div>`: no existe ninguna etiqueta semántica que signifique “grupo de tarjetas”. El `<div>` es un contenedor sin significado, y ese es exactamente el caso en que debe usarse.

```css
.tarjetas {
    display: flex;
    flex-wrap: wrap;
    gap: 1.5rem;
}

.tarjeta {
    flex: 1 1 280px;
    background-color: #ffffff;
    border: 1px solid var(--color-borde);
    border-radius: 6px;
    padding: 1.25rem;
}

.tarjeta h3 {
    color: var(--color-principal);
}
```

La línea clave es `flex: 1 1 280px`, que significa: puede crecer, puede encogerse, y su tamaño de partida es 280 píxeles.

> **Demostración imprescindible.** Reducir el ancho de la ventana del navegador poco a poco. Las tarjetas se encogen hasta llegar a 280 píxeles y entonces **saltan de renglón**, gracias a `flex-wrap: wrap`.
>
> Eso es diseño adaptable sin haber escrito una sola media query. Es el mejor momento para anunciar que en la siguiente sesión se verá Grid y el diseño responsivo completo.

PASO 8 · Tabla, formulario y estados (10 min)

```
table {
    width: 100%;
    border-collapse: collapse;
    margin-bottom: 1.5rem;
}

th {
    background-color: var(--color-principal);
    color: #ffffff;
    text-align: left;
    padding: 0.75rem;
}

td {
    padding: 0.75rem;
    border-bottom: 1px solid var(--color-borde);
}

tbody tr:nth-child(even) {
    background-color: var(--color-claro);
}

fieldset {
    border: 1px solid var(--color-borde);
    border-radius: 6px;
    padding: 1.25rem;
    margin-bottom: 1rem;
}

label {
    display: block;
    margin-bottom: 0.25rem;
    font-weight: bold;
}

input, select, textarea {
    display: block;
    width: 100%;
    max-width: 420px;
    padding: 0.6rem;
    margin-bottom: 1rem;
    border: 1px solid var(--color-borde);
    border-radius: 4px;
    font-family: inherit;
    font-size: 1rem;
}
```

```
        input:focus, select:focus, textarea:focus {
            outline: 2px solid var(--color-principal);
            outline-offset: 1px;
        }

        button {
            background-color: var(--color-principal);
            color: #ffffff;
            border: none;
            border-radius: 4px;
            padding: 0.75rem 1.5rem;
            font-size: 1rem;
            cursor: pointer;
        }

        button:hover {
            background-color: var(--color-oscuro);
        }
```

Tres detalles que conviene señalar:

`border-collapse: collapse` une los bordes de las celdas en una sola línea. Sin esa regla, la tabla se ve con doble borde.

`font-family: inherit` **en los campos.** Los campos de formulario **no heredan** la tipografía del documento: el navegador les impone la suya. Sin esa línea, el formulario se ve con otra letra que el resto de la página.

`:focus` **no se elimina.** Es la marca que indica en qué campo está parado quien navega con el teclado.

> **Demostración.** Recorrer el formulario presionando la tecla *Tab* y observar el recuadro de foco. Después comentar la regla `:focus` y repetir: ya no se sabe dónde está el cursor. Para alguien que no puede usar el ratón, esa página quedó inutilizable. Restaurar la regla.

CIERRE · Lista de verificación (5 min)

#	Criterio	Sí
1	La hoja de estilos está en un archivo externo y vinculada con `<link>`	
2	La hoja comienza con el reinicio y `box-sizing: border-box`	
3	Los colores y espaciados están declarados como variables en `:root`	
4	Se usan clases, no identificadores, para dar estilo	
5	No aparece ningún `!important`	
6	La tipografía se declara en `rem`, no en `px`	
7	El contenido principal tiene ancho máximo y está centrado	
8	La navegación usa Flexbox	
9	Las tarjetas saltan de renglón al reducir la ventana	
10	Los campos del formulario heredan la tipografía del documento	
11	El estado `:focus` es visible al navegar con teclado	
12	La hoja valida sin errores en el validador CSS del W3C	

Errores más comunes y cómo se detectan

Error	Síntoma	Solución
Ruta incorrecta del `<link>`	No cambia absolutamente nada	Revisar carpeta, nombre y minúsculas
Falta el punto y coma	La regla siguiente se ignora	Revisar el bloque anterior al que falla
Falta `box-sizing`	Las columnas no cuadran al agregar padding	Agregar el reinicio
Regla más específica en otro lado	El estilo “no se aplica”	Buscar la regla tachada en las herramientas
Editar solo en el navegador	Los cambios desaparecen al recargar	Escribirlos en el archivo
Tipografía distinta en los campos	El formulario se ve ajeno al resto	`font-family: inherit`
Eliminar el `outline` del foco	La página deja de ser navegable con teclado	No eliminarlo; personalizarlo

---

Ejercicio inmediato (trabajo del estudiante)

Cada estudiante aplica una hoja de estilos externa a la página que construyó en la práctica anterior, con los siguientes requisitos:

- Hoja externa vinculada, con reinicio y `box-sizing: border-box`.
- Paleta de al menos cuatro colores y un espaciado base, declarados como variables en `:root`.
- Tipografía en unidades relativas.
- Contenido principal con ancho máximo y centrado.
- Navegación acomodada con Flexbox.
- Un grupo de al menos tres tarjetas que salten de renglón al reducir la ventana.
- Tabla y formulario con estilo propio, con `:focus` visible.
- Sin identificadores como selectores de estilo y sin `!important`.

Entrega: carpeta comprimida con `index.html`, `css/estilos.css`, la carpeta `img` y capturas del resultado en ventana ancha y ventana angosta.

Lo que sigue

Esta práctica acomodó elementos en **una dimensión** con Flexbox. En la siguiente sesión se trabaja la estructura general de la página con **Grid**, que acomoda en dos dimensiones, y el diseño adaptable con media queries y enfoque de móvil primero.

Anexo · Hoja de estilos completa

El archivo terminado se entrega junto con este documento como `estilos.css`. Igual que en la práctica anterior, conviene distribuirlo **al finalizar la sesión**: si el grupo lo tiene desde el inicio, la demostración se convierte en copiar y pegar.